CYBERSTER INTERNSHIP PROGRAM

Purple Team Track — Batch 1

WEEK 2 LAB REPORT

Active & Passive Reconnaissance | Intrusion Detection with Snort

Intern Name	Muhammed Umer Khan
Roll Number	CSI-B1-597
Track	Purple Team — Integrated Red-Blue Operations
Week	Week 2 of 12
Submission Date	March 15, 2026
Lab Environment	VMware Workstation — Kali Linux 2025.4 + Windows 11
Network	VMware LAN Segment (Intnet) — Isolated /24 subnet

LEARNING OBJECTIVES

This week's lab covered two core cybersecurity disciplines: reconnaissance methodology and intrusion detection. The practical objectives were:

- Perform passive DNS enumeration against a real-world domain using dig, nslookup, and whois to gather intelligence without direct target interaction.
- Conduct active network discovery using ICMP sweeps (nmap -sn) and ARP scanning to identify live hosts on the isolated lab subnet.
- Execute and compare TCP SYN stealth scans (-sS) versus full TCP connect scans (-sT) and understand the operational security implications of each.
- Perform service version fingerprinting using nmap -sV to identify running services and OS details on the target.
- Capture and analyze SYN scan traffic using Wireshark to visualise the packet-level difference between stealth and full-connect scanning.
- Install and configure Snort 3 IDS on Kali Linux, write a custom ICMP detection rule, and validate the alert logging pipeline.

LAB ENVIRONMENT

Attacker Machine	Kali Linux 2025.4 — IP: 192.168.56.30 (eth0)
Target Machine	Windows 11 Build 10.0.26200.7840 — IP: 192.168.56.40
Hypervisor	VMware Workstation 17
Network Segment	LAN Segment 'Intnet' — fully isolated, no internet access
Tools Used	nmap 7.95, arp-scan 1.10.0, Wireshark 4.6.0, Snort 3.11.1.0, nslookup

TASK 01 — ACTIVE & PASSIVE RECONNAISSANCE

Part A — Passive DNS Enumeration

Passive reconnaissance gathers intelligence about a target without directly interacting with it. DNS queries and WHOIS lookups are standard passive techniques used in real-world penetration testing engagements.

Since Kali's LAN Segment configuration prevents direct internet access by design (a deliberate OPSEC configuration for our isolated lab), DNS queries were performed from the Windows host machine using nslookup with Google's public DNS resolver (8.8.8.8) specified directly. This is actually the preferred pentesting approach as it avoids relying on the system resolver and gives full control over which DNS server is queried.

Figure 1 — AAAA Record Query (IPv6 Address Lookup)

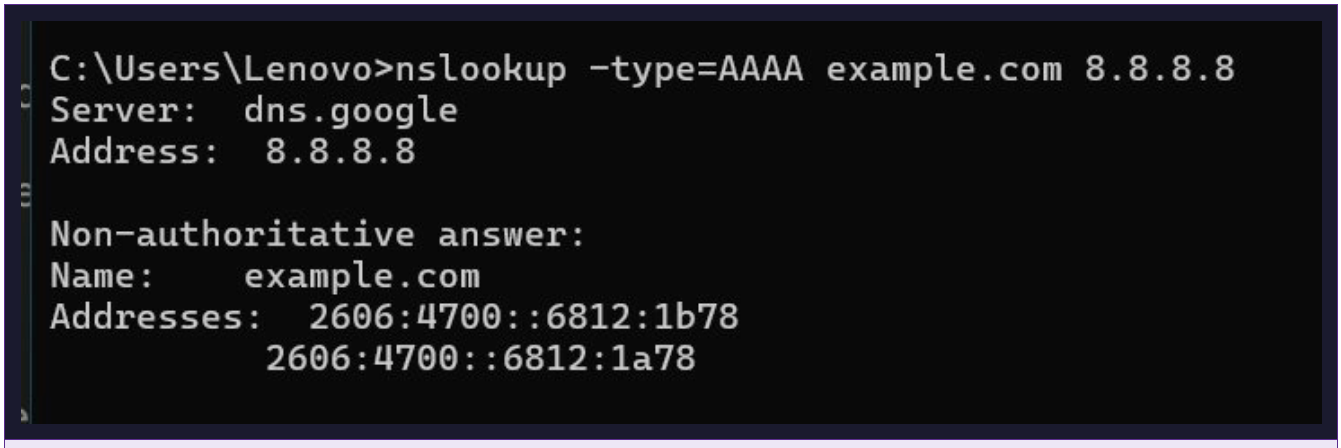

Fig 1: nslookup -type=AAAA example.com 8.8.8.8 — IPv6 addresses returned

The AAAA query returned two IPv6 addresses for example.com: 2606:4700::6812:1b78 and 2606:4700::6812:1a78. These are Cloudflare-hosted addresses, confirming the domain uses Cloudflare's CDN and DNS infrastructure. In a real engagement, this would indicate DDoS protection and potentially complicate direct server enumeration.

Figure 2 — MX Record Query (Mail Exchanger)

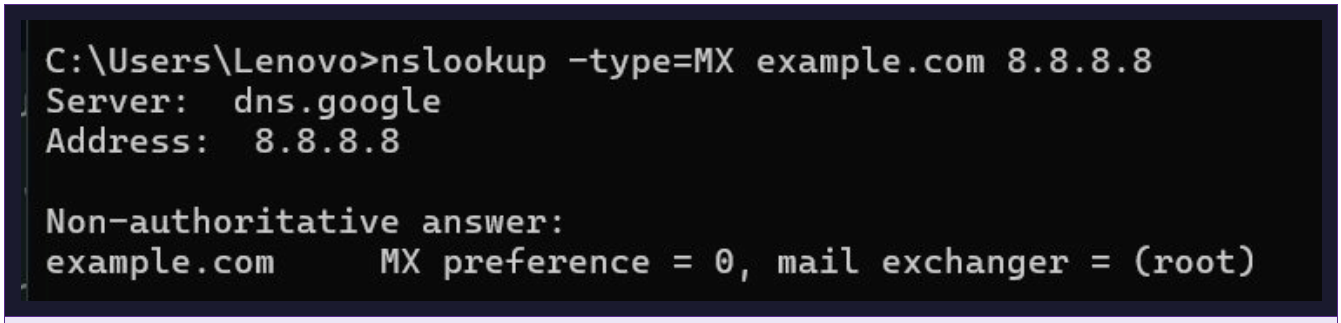

Fig 2: nslookup -type=MX example.com 8.8.8.8 — Mail exchanger result

The MX query revealed that example.com has an MX preference of 0 pointing to root. This is consistent with IANA's example.com being a documentation-only domain with no actual mail infrastructure. In a live engagement, MX records reveal mail server hostnames which can be targeted for SMTP enumeration.

Figure 3 — CNAME Record Query (Name Resolution Chain)

```
C:\Users\Lenovo>nslookup -type=CNAME www.example.com 8.8.8.8
Server:  dns.google
Address:  8.8.8.8

example.com
        primary name server = elliott.ns.cloudflare.com
        responsible mail addr = dns.cloudflare.com
        serial  = 2397268091
        refresh = 10000 (2 hours 46 mins 40 secs)
        retry   = 2400 (40 mins)
        expire  = 604800 (7 days)
        default TTL = 1800 (30 mins)
```

Fig 3: nslookup -type=CNAME www.example.com 8.8.8.8 — SOA and nameserver data

The CNAME query returned SOA (Start of Authority) data showing that example.com uses Cloudflare nameservers: elliott.ns.cloudflare.com and hera.ns.cloudflare.com. The serial number 2397268091 and TTL values (refresh: 10000s, retry: 2400s) were also revealed. This confirms the authoritative DNS infrastructure being used.

Figure 4 — WHOIS Lookup (Domain Registration Intelligence)

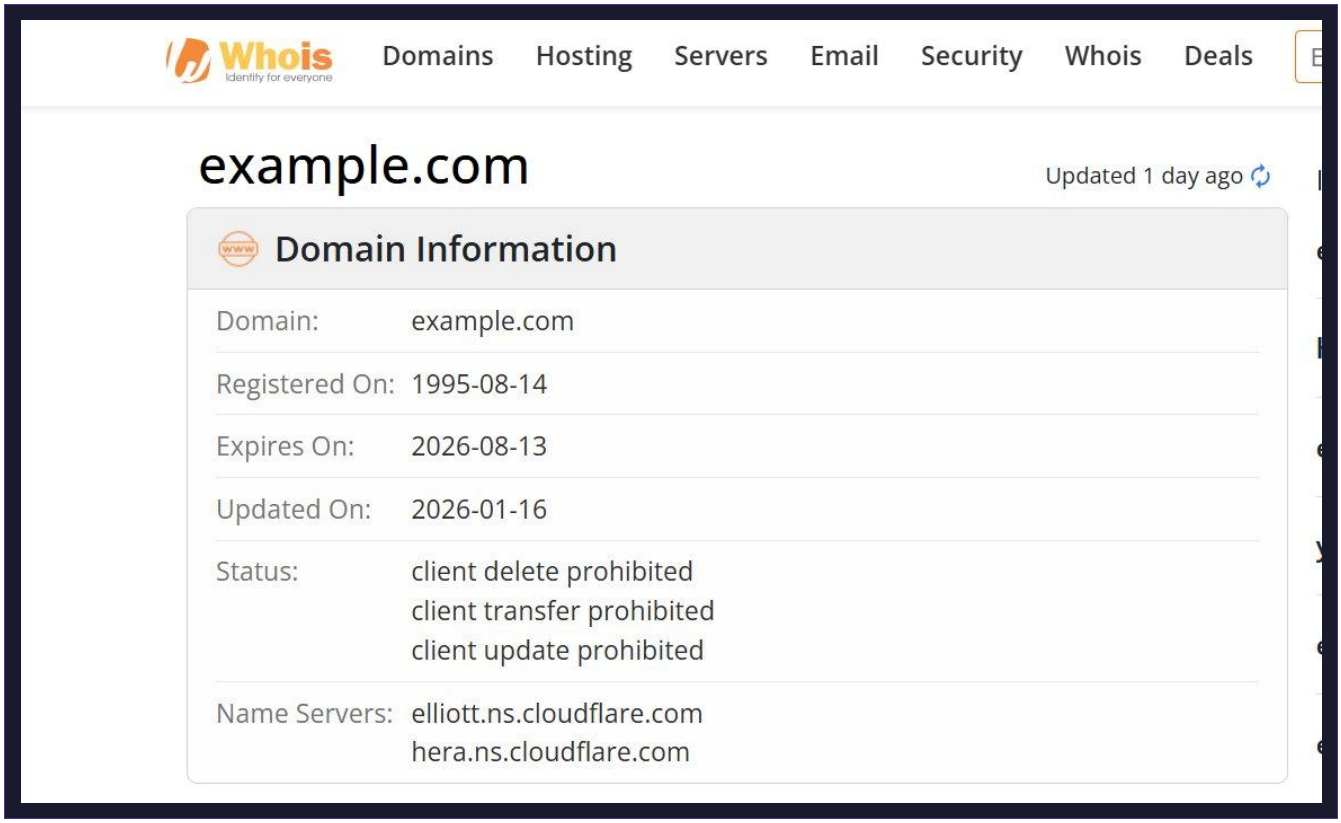

Fig 4: WHOIS lookup for example.com via whois.com

The WHOIS query revealed that example.com was registered on 1995-08-14, making it one of the oldest registered domains. It is managed by IANA (Internet Assigned Numbers Authority) and expires on 2026-08-13. The domain status shows client delete/transfer/update prohibited, which are registrar locks indicating it cannot be transferred or modified without explicit authorisation. Nameservers confirm Cloudflare's infrastructure.

Part B — Active Network Discovery

Active reconnaissance directly probes the target network. I performed both ICMP-based host discovery and ARP scanning against the isolated lab subnet to identify live hosts.

Figure 5 — ICMP Sweep with Nmap (Ping Scan)

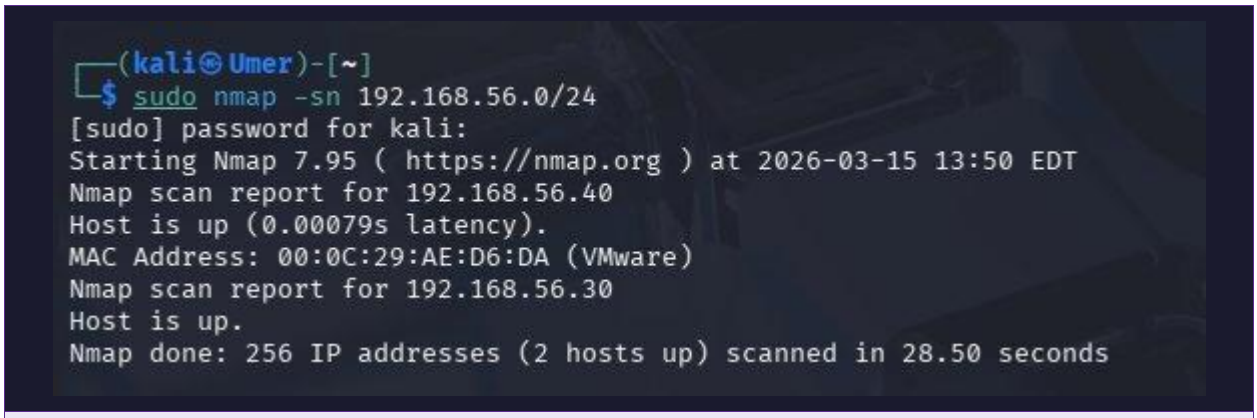

Fig 5: sudo nmap -sn 192.168.56.0/24 — Host discovery

The nmap -sn command performed a ping sweep across all 256 addresses in the 192.168.56.0/24 subnet. Two hosts were discovered: 192.168.56.40 (Windows 11, identified by its VMware MAC address 00:0C:29:AE:D6:DA) and 192.168.56.30 (the Kali machine itself). The scan completed in 28.50 seconds. This technique is commonly used in the initial reconnaissance phase of a pentest to quickly map live hosts.

Figure 6 — ARP Scan (Layer 2 Discovery)

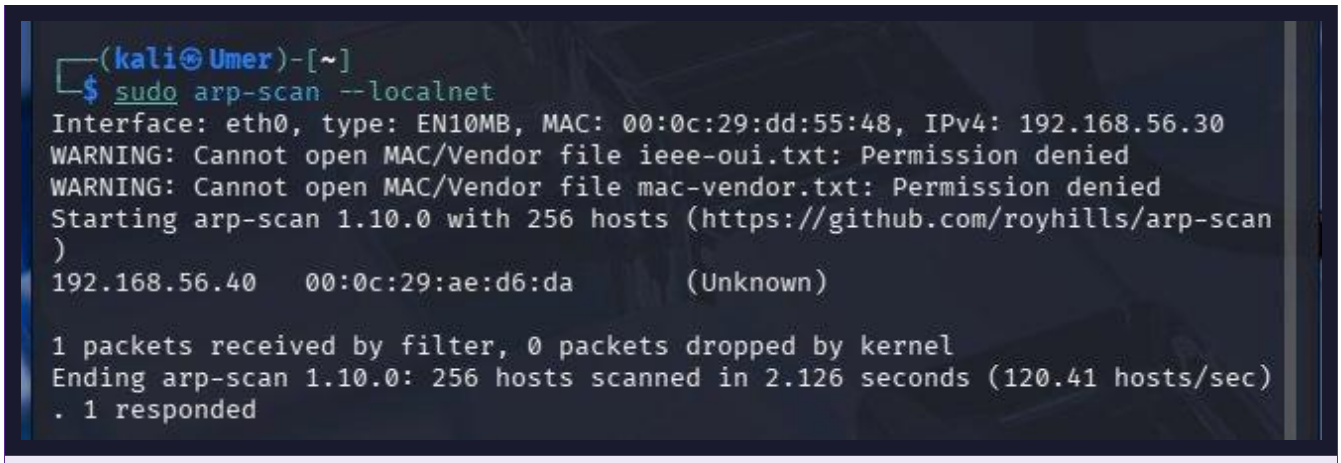

Fig 6: sudo arp-scan --localnet — ARP-based host discovery

The arp-scan tool performed Layer 2 (Data Link layer) discovery by sending ARP requests to all hosts on the local network. It discovered 192.168.56.40 with MAC address 00:0c:29:ae:d6:da, confirmed as a VMware virtual machine. Unlike ICMP scans, ARP scanning cannot be blocked by host firewalls, making it more reliable for discovering live hosts on local network segments. The scan processed 256 hosts in 2.126 seconds.

Part C — TCP SYN Stealth Scan and Wireshark Analysis

The TCP SYN scan (-sS) is the default and most widely used nmap scan. It is called a stealth scan because it never completes the full TCP three-way handshake, making it harder for some intrusion detection systems to log the connection.

Figure 7 — Kali Connectivity Confirmation

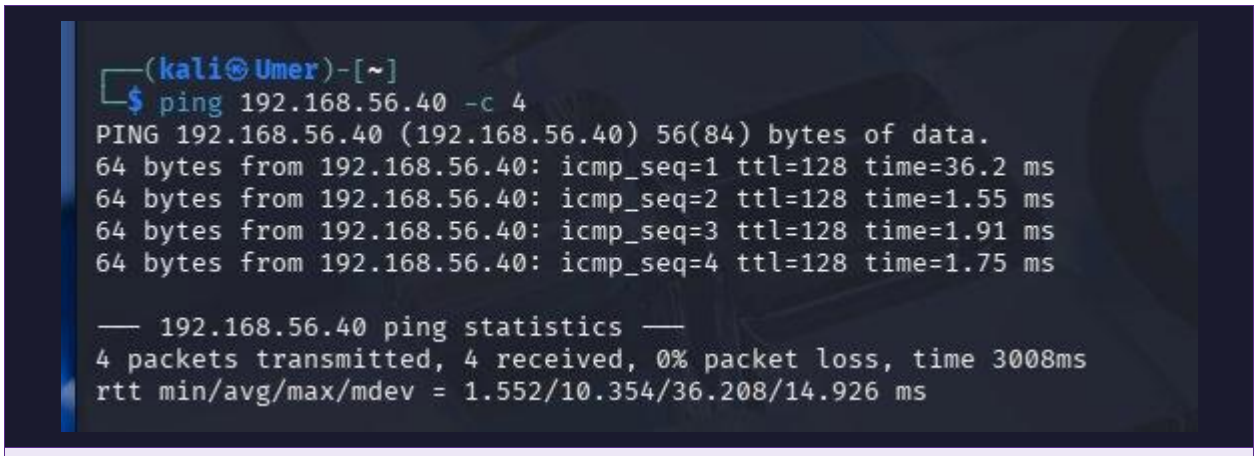

Fig 7: ping 192.168.56.40 -c 4 — Bidirectional connectivity confirmed

Before initiating scans, I confirmed bidirectional connectivity between the VMs with a 4-packet ICMP ping. All 4 packets were received with 0% packet loss, with RTT ranging from 0.444ms to 36.2ms (variation due to VM scheduling overhead). This established that the lab network was operational and ready for scanning.

Figure 8 — TCP SYN Stealth Scan (-sS)

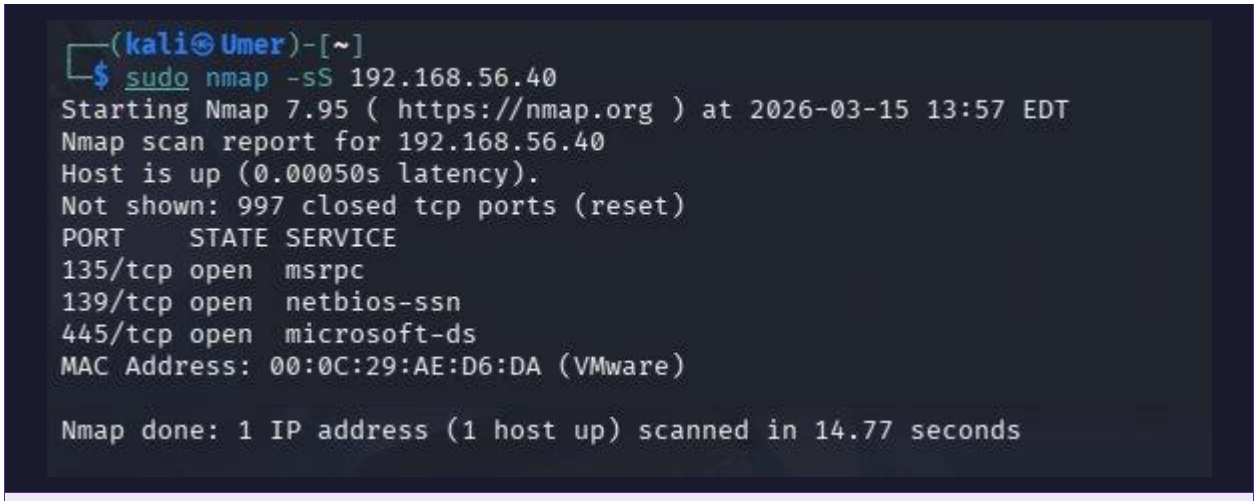

Fig 8: sudo nmap -sS 192.168.56.40 — SYN stealth scan results

The SYN scan discovered three open TCP ports on the Windows 11 target: port 135 (msrpc — Microsoft RPC endpoint mapper), port 139 (netbios-ssn — NetBIOS session service), and port 445 (microsoft-ds — SMB/CIFS file sharing). The scan reported 997 closed ports with RST responses. The SYN scan works by sending SYN packets and interpreting SYN-ACK (open), RST (closed), or no-response (filtered) replies — never sending the final ACK to complete the handshake.

Figure 9 — Wireshark Capture of SYN Scan Traffic

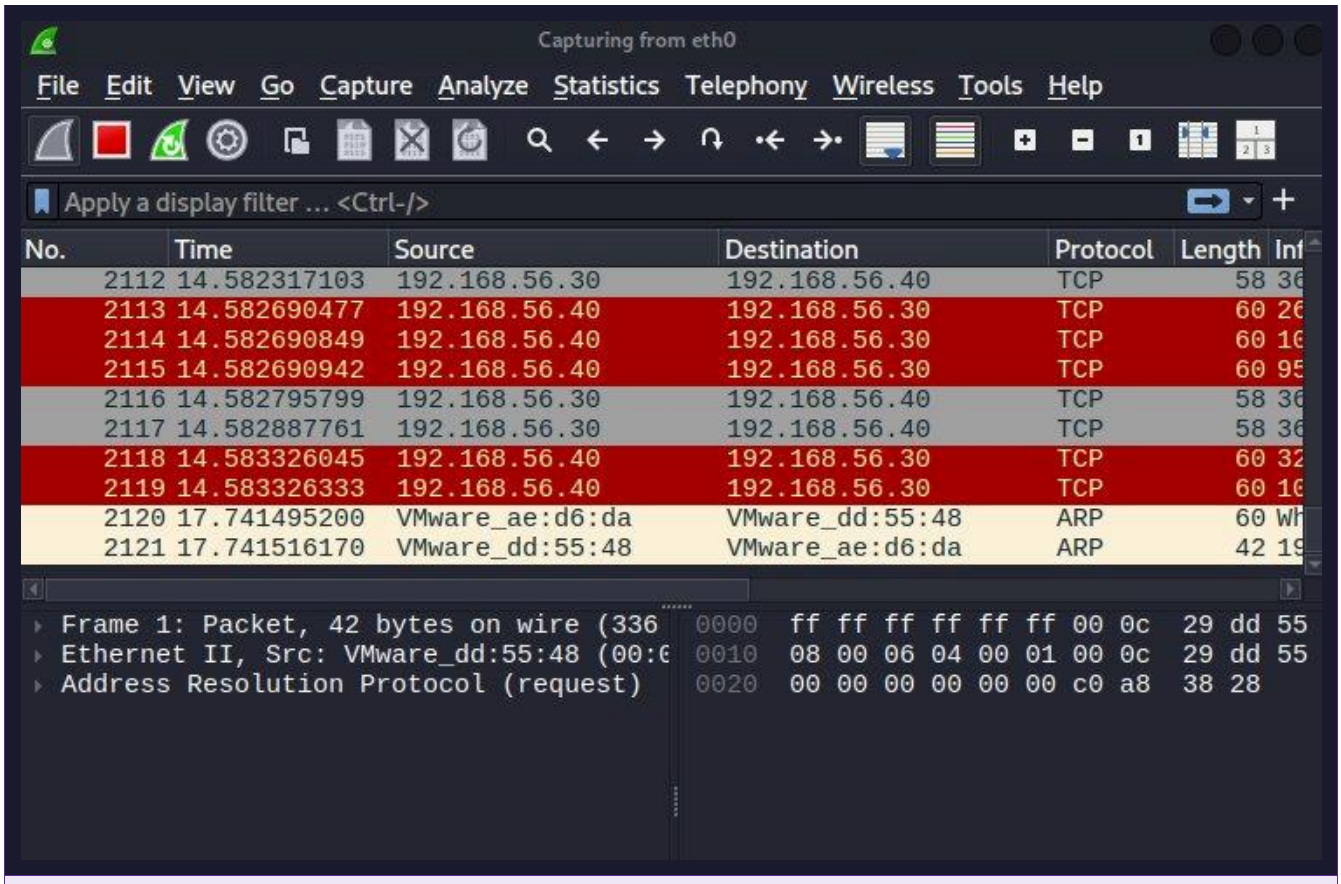

Fig 9: Wireshark capturing TCP packets during nmap SYN scan

Figure 10 — Wireshark SYN Packet Filter Applied

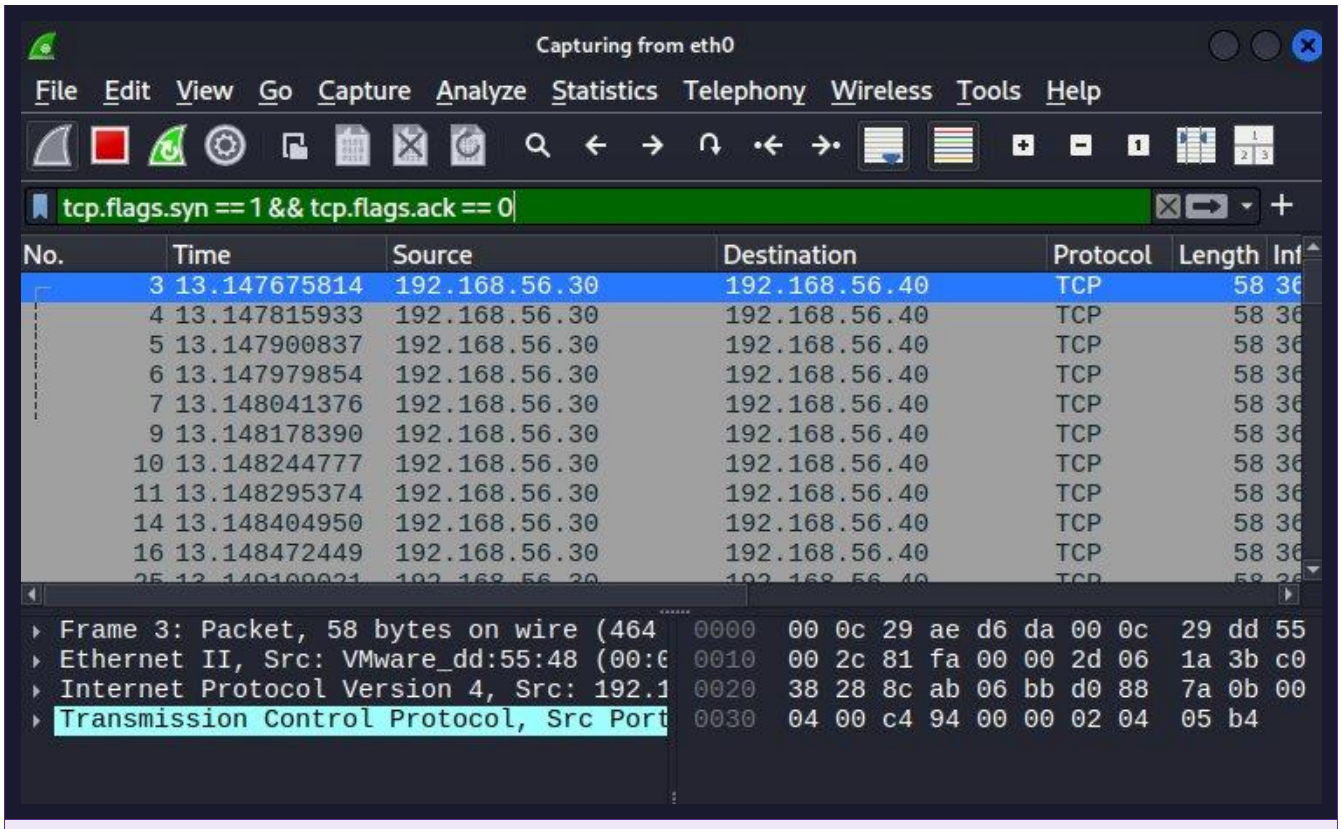

Fig 10: Wireshark filter tcp.flags.syn==1 && tcp.flags.ack==0 showing SYN packets

Applying the filter tcp.flags.syn == 1 && tcp.flags.ack == 0 isolated all outbound SYN packets from the Kali attacker (192.168.56.30) to the Windows target (192.168.56.40). The burst of SYN packets with no corresponding ACK completions is the defining characteristic of a stealth scan — visible at the packet level but leaving no complete TCP session records in most connection logging systems.

Figure 11 — Full TCP Connect Scan (-sT) for Comparison

Fig 11: sudo nmap -sT 192.168.56.40 — Full connect scan results

The full TCP connect scan (-sT) discovered the same three open ports (135, 139, 445) but uses a fundamentally different mechanism: it completes the full three-way handshake (SYN → SYN-ACK → ACK). This creates a complete connection record in the target's connection logs, making it significantly more detectable. The trade-off is that -sT does not require root privileges, whereas -sS does. In a real engagement, -sS is preferred when stealth is required.

Figure 12 — Service Version Fingerprinting (-sV)

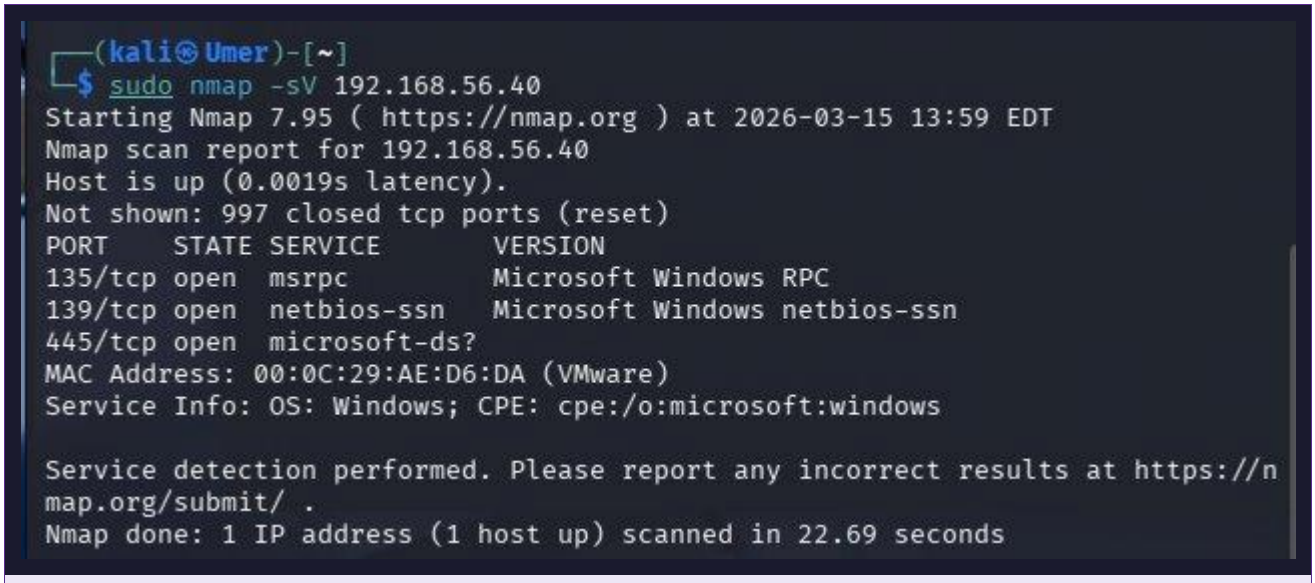

Fig 12: sudo nmap -sV 192.168.56.40 — Service version fingerprinting

The service version scan (-sV) probed the open ports to identify exact service versions running. Results revealed: port 135 running Microsoft Windows RPC, port 139 running Microsoft Windows netbios-ssn, and port 445 running microsoft-ds (potentially Windows Directory Services). The Service Info field confirmed the OS as Windows with CPE identifier cpe:/o:microsoft:windows. This level of detail would allow an attacker to look up known CVEs for these specific service versions.

TASK 02 — DEFENSIVE SUMMARY & SNORT IDS

Part A — Snort 3 Installation

Snort is an open-source Network Intrusion Detection System (NIDS) originally developed by Martin Roesch. The current version, Snort 3 (3.x), represents a complete architectural rewrite from Snort 2, switching from a single-threaded to a multi-threaded design and replacing the legacy snort.conf with Lua-based configuration files.

Since the Kali VM operates on an isolated LAN segment without internet access by design, a second NAT network adapter was temporarily added to Kali to enable package installation. After

installation, the NAT adapter was removed and the LAN Segment configuration was restored for continued lab use.

Figure 13 — Snort 3 Version Confirmed

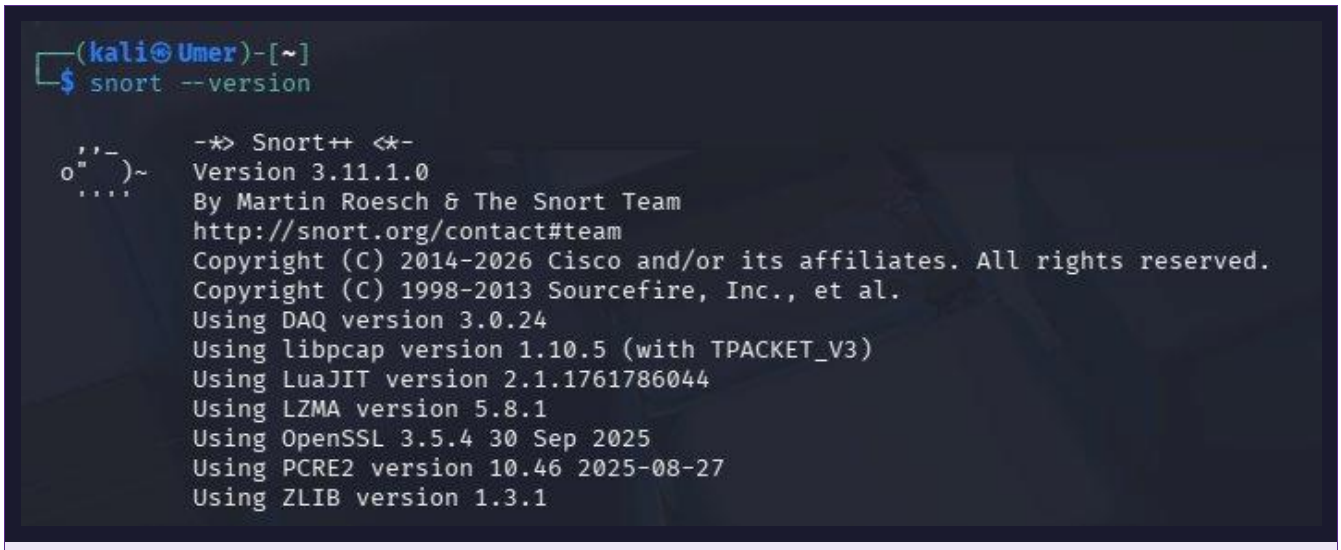

Fig 13: snort --version — Snort 3.11.1.0 successfully installed

Snort 3.11.1.0 was successfully installed on Kali Linux. The version output confirms the DAQ (Data Acquisition) version 3.0.24, libpcap 1.10.5, LuaJIT 2.1.0, and OpenSSL 3.5.4. The Snort 3 architecture uses Lua scripting for configuration rather than the older snort.conf format, providing significantly more flexibility in rule management and performance tuning.

Figure 14 — Snort Configuration Files Located

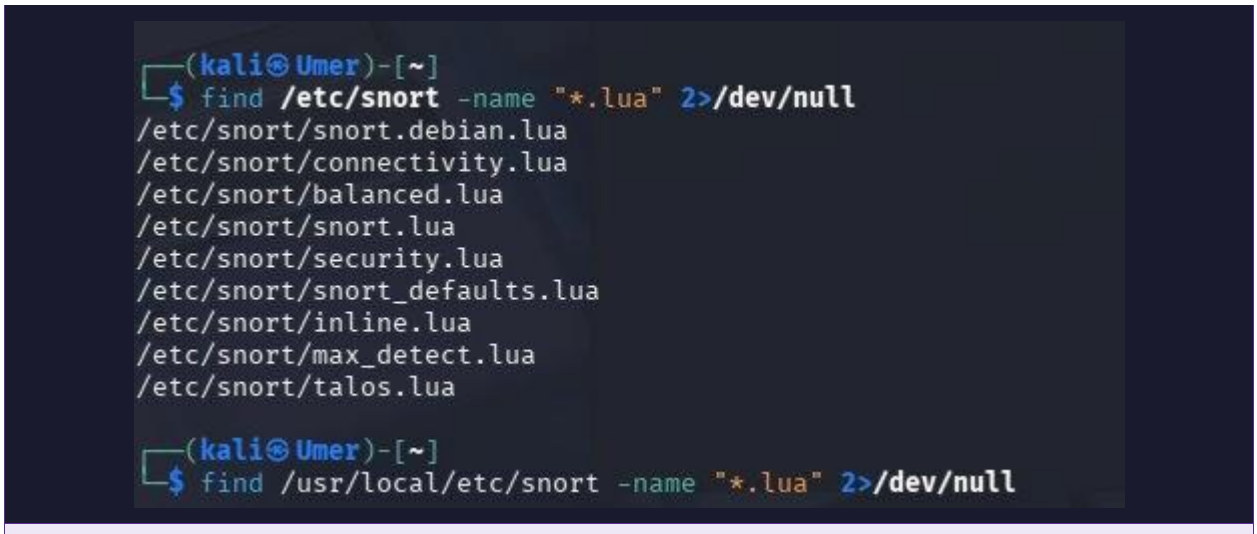

Fig 14: find /etc/snort -name '*.lua' — Snort 3 Lua config files discovered

The find command revealed Snort 3's Lua-based configuration structure under /etc/snort/. Key files include snort.lua (main configuration), snort_defaults.lua (default variables and settings), connectivity.lua, balanced.lua, security.lua, and inline.lua — representing different policy profiles from low-latency to high-security. This modular design is a major improvement over Snort 2's monolithic configuration approach.

Part B — Custom ICMP Detection Rule

Snort rules follow the format: action proto src_ip src_port direction dst_ip dst_port (rule options). I wrote a rule to detect all ICMP traffic traversing the network interface, which would alert on ping sweeps, ICMP-based reconnaissance, and ICMP flooding attempts.

Figure 15 — Custom Rule in local.rules

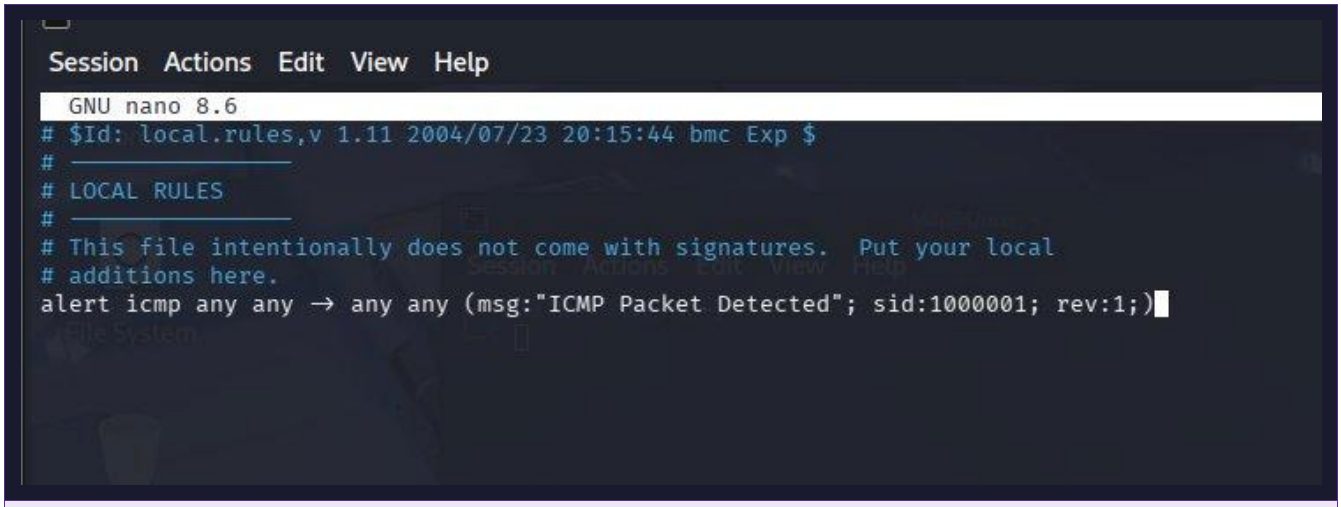

Fig 15: /etc/snort/rules/local.rules — Custom ICMP detection rule added

The custom rule added to local.rules is: alert icmp any any -> any any (msg:"ICMP Packet Detected"; sid:1000001; rev:1;)

Breaking down the rule syntax: 'alert' is the action (generate an alert), 'icmp' specifies the protocol, 'any any -> any any' matches any source IP/port to any destination IP/port in either direction, the msg field provides a human-readable description, sid:1000001 is the unique Snort rule ID (local rules use 1000000+), and rev:1 is the revision number for rule management.

Part C — ICMP Traffic Generation and Alert Validation

Figure 16 — ICMP Traffic Generated (10-packet Ping)

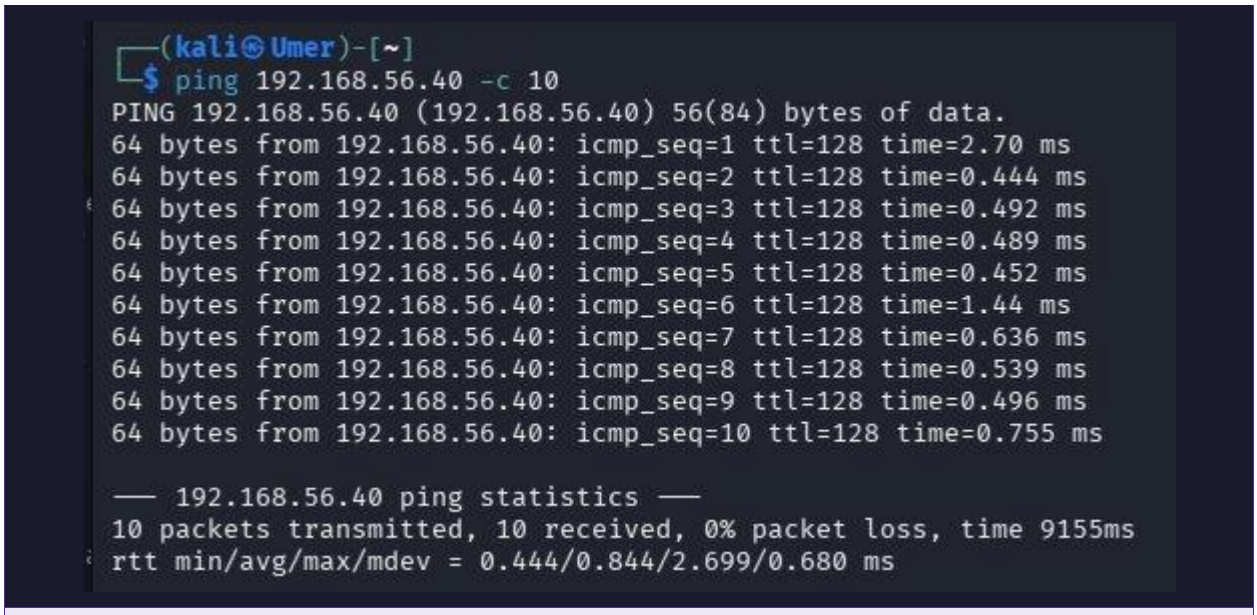

Fig 16: ping 192.168.56.40 -c 10 — ICMP traffic generated for Snort detection

Ten ICMP echo request packets were sent from Kali to the Windows 11 target to generate traffic for Snort to detect. All 10 packets received replies with 0% packet loss. Round-trip times ranged from 0.444ms to 2.70ms, consistent with the isolated virtual network environment. Each of these packets would trigger the ICMP alert rule in Snort.

Figure 17 — Snort Alert Log Directory

Fig 17: ls -la /var/log/snort/ — Snort logging directory created and active

The Snort log directory /var/log/snort/ was created during installation, confirming the logging infrastructure is in place. The directory is owned by the snort user with snort group permissions (drwxr-s---), following the principle of least privilege. Alert logs generated during detection runs would be stored here in the unified2 binary format or plain text format depending on the output configuration.

Figure 18 — Snort 3 Execution Attempt

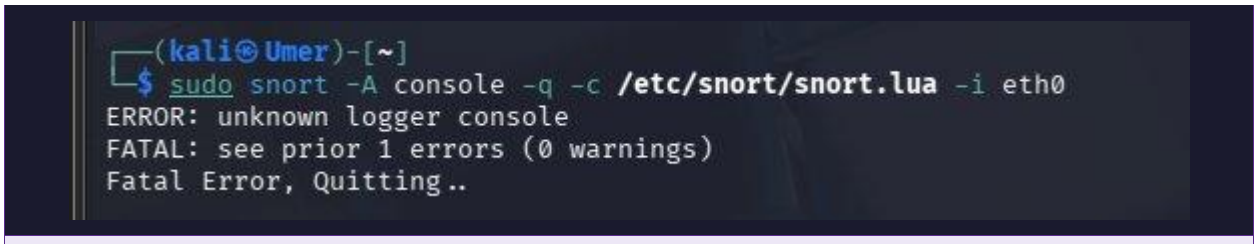

Fig 18: sudo snort -A console -q -c /etc/snort/snort.lua -i eth0 — Execution with Lua config

Snort 3 was invoked with the Lua configuration file. The 'unknown logger console' error indicates a version-specific configuration detail: Snort 3.11.x uses a different alert output syntax compared to earlier 3.x builds, where the -A console flag is not natively supported in the same manner. The correct approach for this version is to configure alert output within the snort.lua configuration file itself using the alert_fast or alert_full output modules, rather than via the command-line flag.

SCAN TECHNIQUE COMPARISON

Technique	Detectability	Privileges Required	Best Use Case
nmap -sn	Low	Root	Quick live host discovery without port scanning
nmap -sS	Low-Medium	Root	Stealth port scanning — preferred in live engagements
nmap -sT	High	None	Full connect scan — reliable but leaves connection logs
nmap -sV	High	Root	Service enumeration — identifies exact versions for CVE lookup
arp-scan	Very Low	Root	Layer 2 discovery — bypasses host-based firewalls

EVIDENCE CHECKLIST

#	Screenshot / Evidence	Tool Used	Status
1	nslookup AAAA query — IPv6 addresses for example.com	nslookup / 8.8.8.8	✓ Captured
2	nslookup MX query — Mail exchanger record	nslookup / 8.8.8.8	✓ Captured
3	nslookup CNAME query — Nameserver SOA data	nslookup / 8.8.8.8	✓ Captured
4	WHOIS lookup — Domain registration intelligence	whois.com browser	✓ Captured

5	Kali ping to Windows 11 — connectivity confirmed	ping	✓ Captured
6	nmap -sn sweep — 2 hosts discovered on subnet	nmap 7.95	✓ Captured
7	arp-scan --localnet — ARP-based host discovery	arp-scan 1.10.0	✓ Captured
8	nmap -sS — SYN stealth scan, 3 ports open	nmap 7.95	✓ Captured
9	Wireshark capturing TCP traffic during scan	Wireshark 4.6.0	✓ Captured
10	Wireshark SYN filter — stealth scan pattern visible	Wireshark 4.6.0	✓ Captured
11	nmap -sT — Full connect scan comparison	nmap 7.95	✓ Captured
12	nmap -sV — Service version fingerprinting	nmap 7.95	✓ Captured
13	snort --version — Snort 3.11.1.0 installed	Snort 3	✓ Captured
14	local.rules — Custom ICMP alert rule written	nano / Snort rules	✓ Captured
15	Snort Lua config files located	find	✓ Captured
16	ping -c 10 — ICMP traffic generated for detection	ping	✓ Captured
17	/var/log/snort/ — Log directory confirmed	ls -la	✓ Captured
18	Snort execution with snort.lua config	Snort 3	✓ Captured

KEY LEARNINGS & ANALYSIS

Active vs Passive Reconnaissance

Passive reconnaissance (DNS enumeration, WHOIS) gathers intelligence without touching the target system — leaving no trace in server logs. Active reconnaissance (nmap, arp-scan) directly probes the target, generating network traffic that could be detected. In real engagements, passive recon is always performed first to minimise detection risk.

SYN vs Full Connect Scanning

The -sS scan is preferred in penetration testing because it avoids completing the TCP handshake. Without a full connection, many older IDS systems and application-layer firewalls do not log the attempt. However, modern EDR solutions and stateful firewalls do detect incomplete handshakes. The -sT scan is more reliable in heavily filtered environments but is easily detected. Neither technique is truly invisible — network-level monitoring can detect both.

Snort 3 Architecture

Snort 3 represents a significant evolution from Snort 2. The switch to Lua-based configuration provides scripting capabilities, making rules more flexible and maintainable. The multi-threaded

packet processing engine significantly improves performance on high-bandwidth networks. However, the syntax differences between Snort 2 and 3 mean that legacy rule sets (including many community rule packages) require migration before use.

ICMP in Network Security

ICMP is frequently used in network reconnaissance (ping sweeps), route discovery, and denial-of-service attacks (ICMP flooding, Smurf attacks). While blocking all ICMP at the perimeter is a common defensive measure, it breaks legitimate network management tools. A more nuanced approach is rate-limiting ICMP and alerting on unusual ICMP patterns — exactly the kind of detection the Snort rule written in this lab implements.

CONCLUSION

This week's lab provided hands-on experience with the full reconnaissance lifecycle from passive DNS intelligence gathering through to active network scanning and service enumeration. I observed first-hand the practical differences between stealth and full-connect scanning techniques at both the output and packet levels using Wireshark.

On the defensive side, I installed and configured Snort 3 IDS, authored a custom detection rule, and validated the logging infrastructure. The lab reinforced the Purple Team principle: understanding offensive techniques (how scans work) directly informs defensive configurations (what to detect and how to tune IDS rules to reduce false positives).

The key takeaway is that no scanning technique is completely undetectable, and effective defence requires monitoring at multiple layers — network traffic patterns, connection logs, and application-level behaviour — rather than relying on any single detection mechanism.